

Otimização Dinâmica de Rotas para Coleta de Resíduos Sólidos: Adaptação do Algoritmo de Solomon com Integração IoT

1 Introdução

A gestão de resíduos sólidos urbanos impõe desafios operacionais, econômicos e ambientais aos serviços municipais. Sistemas convencionais de coleta, baseados em rotas fixas e frequências predeterminadas, podem gerar deslocamentos desnecessários e dificultar respostas adequadas a situações de enchimento crítico dos recipientes (Pardini et al., 2020; Lozano et al., 2018). A literatura aponta que a integração entre monitoramento em tempo real e técnicas de otimização pode tornar o planejamento da coleta mais eficiente (Lozano et al., 2018; Barth et al., 2023).

Este subprojeto, vinculado ao projeto “Pesquisa Operacional e Inovação em Logística Regional”, propõe uma solução integrada que combina sensores IoT via LoRaWAN (Ramson et al., 2021; Almeida and Borin, 2021) com heurísticas baseadas em Solomon (1987) para o Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW), além do uso de um Digital Twin para apoio à simulação e à visualização das rotas (Barth et al., 2023). A pergunta de pesquisa que orienta o trabalho é: *de que modo a integração entre monitoramento IoT e heurísticas clássicas para o VRPTW pode tornar o planejamento da coleta mais responsivo e eficiente, preservando a factibilidade temporal das rotas?*

1.1 Definição Formal do Problema

O VRPTW é definido sobre um grafo completo $G = (V, E)$, onde $V = \{0, 1, \dots, n\}$ é o conjunto de nós, sendo 0 o depósito e $\{1, \dots, n\}$ os clientes. Cada arco $(i, j) \in E$ tem custo d_{ij} (distância euclidiana). Cada cliente i possui demanda q_i , janela temporal $[e_i, l_i]$, tempo de serviço s_i e coordenadas (x_i, y_i) . A frota é homogênea com capacidade Q ; o número de veículos K não é limitado *a priori*. Toda rota inicia e termina em 0.

Restrições: (i) cobertura: cada cliente é visitado exatamente uma vez; (ii) capacidade: $\sum_{i \in R} q_i \leq Q$ para cada rota R ; (iii) janelas: o serviço em i inicia em t_i com $e_i \leq t_i \leq l_i$ (espera permitida se a chegada for anterior a e_i); (iv) precedência: 0 é o primeiro e o último nó de cada rota.

Função objetivo: duas convenções consagradas são adotadas em paralelo neste trabalho (ver Seção 3.2):

$$\min \sum_{k=1}^K \sum_{(i,j) \in R_k} d_{ij} \quad (1)$$

no caso DIMACS, ou a hierarquia lexicográfica $(K, \sum d_{ij})$ no caso Solomon clássico.

2 Revisão Bibliográfica

2.1 Métodos Construtivos: Solomon I1

Solomon (1987) introduziu uma família de heurísticas de inserção sequencial que se tornaram referência para o VRPTW. A I1 constrói rotas iterativamente: seleciona um cliente-semente (em nossa implementação, o mais distante do depósito) e insere os demais clientes na posição que minimiza um critério ponderado C_1 , dentre os candidatos selecionados pelo critério C_2 . Os componentes c_{11} (custo geométrico) e c_{12} (impacto temporal) controlam, respectivamente, o aumento de distância e o atraso causado pela inserção.

2.2 Busca Local e Variable Neighborhood Descent (VND)

A busca local explora sistematicamente vizinhanças de uma solução aplicando operadores de modificação. Os operadores clássicos para VRPTW (Bräysy and Gendreau, 2005; Toth and Vigo, 2003) incluem *Relocate* (Shaw, 1998), *Swap*, *2-opt* (Lin and Kernighan, 1973), *2-opt** (entre rotas) (Potvin and Rousseau, 1995), *Or-opt* (Or, 1976) e *Cross-exchange*. Toth and Vigo (2003) propõem a vizinhança granular, restringindo a busca aos k clientes mais próximos para reduzir o custo computacional.

O VND (Hansen and Mladenović, 2001) é um esquema de descida que percorre uma lista ordenada de vizinhanças $\mathcal{N}_1, \dots, \mathcal{N}_\ell$: ao encontrar melhoria em $\mathcal{N}_i$ retorna a $\mathcal{N}_1$; caso contrário avança para $\mathcal{N}_{i+1}$. Termina em ótimo local com respeito a todas as vizinhanças. Melo et al. (2008) aplicaram VND ao VRPTW combinando vizinhanças intra e inter-rota.

2.3 GRASP: Variantes Fixa e Reativa

O GRASP (Feo and Resende, 1995; Resende and Ribeiro, 2003) é uma metaheurística multi-partida que alterna construção gulosa-randomizada e busca local. A construção usa uma Lista Restrita de Candidatos (RCL) parametrizada por $\alpha \in [0, 1]$:

$$\text{RCL}(\alpha) = \{c \in N \mid \text{custo}(c) \leq c_{\min} + \alpha(c_{\max} - c_{\min})\}, \quad (2)$$

onde $c_{\min}, c_{\max}$ são o menor e o maior custo de inserção entre os candidatos. Valores pequenos de α tendem a comportamento guloso; valores altos, a maior diversificação.

A variante *reativa* (Prais and Ribeiro, 2000) mantém um conjunto $\alpha = (\alpha_1, \dots, \alpha_m)$ e probabilidades $p_1, \dots, p_m$ atualizadas a cada bloco de iterações em função da qualidade média obtida com cada α_i , ajustando automaticamente o balanço intensificação–diversificação.

2.4 Busca Tabu

Glover (1989, 1990) introduziram a Busca Tabu como metaheurística que mantém uma memória de curto prazo (lista tabu) para evitar ciclos. Cordeau et al. (2001) consolidaram uma implementação unificada para o VRPTW. Os elementos centrais são: (i) atributos tabu (clientes ou arcos modificados) com tempo de expiração (*tenure*); (ii) critério de aspiração, que admite movimentos tabu se levarem a uma solução melhor que a melhor já encontrada; (iii) políticas de seleção (*best-improvement* ou *first-improvement*). A discussão detalhada do uso de memória de curto prazo encontra-se em Glover and Laguna (1997) e Gendreau and Potvin (2010).

2.5 Monitoramento IoT e Digital Twin para Coleta

A literatura recente combina sensores de nível em recipientes, comunicação LPWAN (LoRaWAN) e modelos de decisão para tornar a coleta orientada por demanda. Lozano et al. (2018) propõem um sistema baseado em LoRaWAN com otimização de rotas; Ramson et al. (2021) apresentam um sistema de monitoramento de nível de lixeiras com IoT–LoRaWAN; Pardini et al. (2020) discutem soluções orientadas ao cidadão; Almeida and Borin (2021) descrevem uma plataforma sustentável baseada em IoT e LPWAN; Barth et al. (2023) integram esses elementos em um *Digital Twin* de apoio à decisão para gestão de resíduos.

3 Metodologia

3.1 Notação e Arquitetura do Solver

A notação adotada nos pseudocódigos está resumida no Quadro 1. O solver foi implementado em C++20 e organiza-se em quatro fases que podem ser executadas isoladamente ou compostas: (1) construção por Solomon

I1, (2) busca local por VND, (3) GRASP (fixo ou reativo) com VND como melhoria, (4) Busca Tabu com a mesma vizinhança granular do VND.

Quadro 1: Notação utilizada nos pseudocódigos

Símbolo	Significado
$I = (V, d, Q, q, e, l, s)$	instância VRPTW
S	solução (conjunto de rotas)
$R = [0, c_1, \dots, c_r, 0]$	rota individual
N	conjunto de clientes ainda não roteados
$\mathcal{N}_k$	k -ésima vizinhança (operador)
$\mathcal{T}$	lista tabu (atributo $\rightarrow$ iteração de expiração)
S^*	melhor solução encontrada (incumbente)
δ	variação de custo da aplicação de um movimento
α	parâmetro da RCL no GRASP
T	<i>tenure</i> (duração tabu)
$f(S)$	valor objetivo de S no critério adotado (§3.2)
$\prec$	relação “melhor que” no critério adotado (lex. ou DIMACS)

3.2 Critério de Avaliação e Referência de Comparação

Duas convenções coexistem no VRPTW e foram ambas implementadas no solver. Na convenção clássica (Solomon, 1987), o objetivo é lexicográfico — primeiro minimiza-se o número de veículos K , depois a distância total $\sum d_{ij}$ —, com distâncias arredondadas em duas casas decimais. Na convenção do 12º DIMACS Implementation Challenge (Golden et al., 2008), o objetivo é a distância total como critério único, com uma casa decimal. A coexistência das duas convenções implica diferenças relevantes na comparação com Best-Known Solutions (BKS): uma solução ótima sob um critério pode ser subótima sob o outro.

Histórico do repositório de BKS. Os experimentos iniciais deste trabalho utilizavam o repositório de melhores soluções mantido pela equipe *Gehring & Homberger* no âmbito do 12º DIMACS Challenge (Gehring and Homberger, 1999; Golden et al., 2008), que reunia BKS sob o critério DIMACS. Esse repositório tornou-se indisponível durante a execução do projeto. Como alternativa, adotamos as soluções publicadas no sítio pessoal de M. M. Solomon¹ para as instâncias de 100 clientes, que correspondem ao critério clássico (lexicográfico veículos–distância, duas casas decimais). Por essa razão o solver mantém as duas implementações de objetivo: o critério DIMACS para comparações futuras com a literatura recente do desafio e o critério clássico para confronto direto com as BKS atualmente disponíveis. Quando os dois critérios divergem em uma instância, ambos os valores são reportados.

3.3 Fase 1: Construção por Solomon I1

A solução inicial é gerada pela heurística de inserção Solomon I1 (Algoritmo 1). O algoritmo constrói rotas sequencialmente: para cada nova rota seleciona o cliente u^* mais distante do depósito como semente (linha 5) e a satura inserindo iterativamente o cliente não roteado que maximiza o critério C_2 na posição que minimiza C_1 (sub-rotina *MelhorInserção*, linha 8).

¹<http://w.cba.neu.edu/~msolomon/problems.htm>

Algoritmo 1: Solomon I1 (construção por inserção)

Entrada: Instância I , parâmetros $\mu, \alpha_1, \alpha_2, \lambda$ **Saída:** Solução inicial S

```
1  $S \leftarrow \emptyset$ ;  
2  $N \leftarrow \{1, \dots, n\}$ ;  
3 while  $N \neq \emptyset$  do  
4    $u^* \leftarrow \text{SelecionarSemente}(N, d)$ ; // cliente mais distante de 0  
5    $R \leftarrow [0, u^*, 0]$ ;  $N \leftarrow N \setminus \{u^*\}$ ;  
6   repeat  
7      $(u, p) \leftarrow \text{MelhorInserção}(N, R, I)$ ;  
8     if  $u \neq \text{nulo}$  then  
9       inserir  $u$  em  $R$  na posição  $p$ ;  
10       $N \leftarrow N \setminus \{u\}$ ;  
11    end if  
12  until nenhuma inserção viável;  
13   $S \leftarrow S \cup \{R\}$ ;  
14 end while  
15 return  $S$ ;
```

A sub-rotina de melhor inserção avalia, para cada $u \in N$ e cada par de nós adjacentes (i, j) em R , a viabilidade simultânea de capacidade e janelas, e seleciona o par com menor C_1 . Em seguida, dentre os clientes viáveis, retorna o que maximiza C_2 (Algoritmo 2).

Algoritmo 2: MelhorInserção — Sub-rotina I1

Entrada: N , rota R , instância I **Saída:** Cliente u^* e posição p^* (ou nulo se ineficaz)

```
1  $u^* \leftarrow \text{nulo}$ ;  $C_2^* \leftarrow -\infty$ ;  
2 foreach  $u \in N$  do  
3    $C_1^{(u)} \leftarrow +\infty$ ;  $p^{(u)} \leftarrow \text{nulo}$ ;  
4   foreach posição  $(i, j)$  em  $R$  do  
5     if inserção  $(i, u, j)$  viável em capacidade e janelas then  
6        $c_{11} \leftarrow d_{iu} + d_{uj} - \mu d_{ij}$ ; // Eq. 3  
7        $c_{12} \leftarrow t_j^{\text{novo}} - t_j^{\text{atual}}$ ; // Eq. 4  
8        $C_1 \leftarrow \alpha_1 c_{11} + \alpha_2 c_{12}$ ;  
9       if  $C_1 < C_1^{(u)}$  then  
10         $C_1^{(u)} \leftarrow C_1$ ;  $p^{(u)} \leftarrow (i, j)$ ;  
11      end if  
12    end if  
13  end foreach  
14  if  $p^{(u)} \neq \text{nulo}$  then  
15     $C_2 \leftarrow \lambda d_{0u} - C_1^{(u)}$ ;  
16    if  $C_2 > C_2^*$  then  
17       $u^* \leftarrow u$ ;  $p^* \leftarrow p^{(u)}$ ;  $C_2^* \leftarrow C_2$ ;  
18    end if  
19  end if  
20 end foreach  
21 return  $(u^*, p^*)$ ;
```

$$c_{11}(i, u, j) = d_{iu} + d_{uj} - \mu \cdot d_{ij} \quad (3)$$

$$c_{12}(i, u, j) = t_j^{\text{nov}} - t_j^{\text{atual}} \quad (4)$$

Os parâmetros adotados são os clássicos: $\mu = 1,0$, $\alpha_1 = 1,0$, $\alpha_2 = 0,0$, $\lambda = 2,0$ (Solomon, 1987). A viabilidade temporal usa análise de *forward slack*: a folga máxima admissível em cada posição é calculada para garantir que nenhuma janela posterior seja violada.

3.4 Fase 2: VND (Variable Neighborhood Descent)

A busca local adotada (Algoritmo 3) segue o esquema VND (Hansen and Mladenović, 2001) com política *first-improvement* dentro de cada vizinhança. Ao encontrar melhoria em $\mathcal{N}_k$, retorna-se a $\mathcal{N}_1$; caso contrário, avança-se para $\mathcal{N}_{k+1}$. A ordem padrão das vizinhanças foi definida após análise de contribuição individual (§3.7): RelocateIntra $\rightarrow$ SwapIntra $\rightarrow$ RelocateInter $\rightarrow$ 2-opt*. O critério de melhoria é lexicográfico no par (custo, K): para custos praticamente iguais (tolerância $\varepsilon = 10^{-6}$), prefere-se a solução com menos rotas.

Algoritmo 3: VND — Variable Neighborhood Descent

Entrada: Solução S , lista ordenada $(\mathcal{N}_1, \dots, \mathcal{N}_\ell)$

Saída: Ótimo local S em relação a todas as vizinhanças

```

1  $k \leftarrow 1$ ;
2 while  $k \leq \ell$  do
3    $S' \leftarrow$  primeiro vizinho de  $S$  em  $\mathcal{N}_k$  com  $f(S') \prec f(S)$ ;           // first-improvement
4   if  $S' \neq \text{nulo}$  then
5      $S \leftarrow S'$ ;  $k \leftarrow 1$ ;                                     // melhoria  $\Rightarrow$  reinicia da  $\mathcal{N}_1$ 
6   else
7      $k \leftarrow k + 1$ ;                                             // exauriu  $\mathcal{N}_k$ 
8   end if
9 end while
10 return  $S$ ;
```

A enumeração de vizinhos usa avaliação incremental: somente o Δ (variação de custo) é calculado, e a vizinhança é granular (Toth and Vigo, 2003), restrita aos $k = 20$ clientes mais próximos por padrão, reduzindo o esforço por iteração de $\mathcal{O}(n^2)$ para $\mathcal{O}(nk)$. Para o operador Relocate, por exemplo:

$$\Delta_{\text{relocate}} = \underbrace{[d_{x,u} + d_{u,y} - d_{x,y}]}_{\text{inserção em } R_2} + \underbrace{[d_{i,j} - d_{i,u} - d_{u,j}]}_{\text{remoção de } R_1}. \quad (5)$$

3.5 Fase 3: GRASP Fixo e Reativo

O GRASP (Algoritmo 4) executa, a cada iteração, uma construção gulosa-randomizada seguida de VND. A construção (Algoritmo 5) reaproveita a estrutura I_1 , substituindo a seleção gulosa do candidato por uma escolha uniforme dentro da RCL definida pela Eq. 2.

Algoritmo 4: GRASP (fixo e reativo) com VND como busca local

Entrada: Instância I , modo $M \in \{\text{fixo}, \text{reativo}\}$, α ou conjunto α

Parâmetros: Número máximo de iterações N_{it} , bloco reativo b , γ

Saída: Melhor solução S^*

```
1  $S^* \leftarrow \text{Solomon I1}(I);$  // incumbente determinístico
2 if  $M = \text{reativo}$  then
3    $p_i \leftarrow 1/|\alpha|$  para todo  $i$ ;
4 end if
5 for  $k \leftarrow 1$  até  $N_{it}$  do
6   if  $M = \text{fixo}$  then
7      $\alpha_k \leftarrow \alpha$ ;
8   else
9     sortear índice  $i$  com probabilidade  $p_i$ ;  $\alpha_k \leftarrow \alpha_i$ ;
10  end if
11   $S \leftarrow \text{ConstruçãoRandomizada}(I, \alpha_k);$ 
12   $S \leftarrow \text{VND}(S);$ 
13  if  $f(S) \prec f(S^*)$  then
14     $S^* \leftarrow S$ ;
15  end if
16  if  $M = \text{reativo}$  e  $k \bmod b = 0$  then
17     $p \leftarrow \text{AtualizarProbabilidades}(p, \alpha, S^*, \gamma);$  // Eq. 6
18  end if
19 end for
20 return  $S^*$ ;
```

Algoritmo 5: ConstruçãoRandomizada — construção com RCL

Entrada: Instância I , parâmetro α

Saída: Solução S

```
1  $S \leftarrow \emptyset$ ;  $N \leftarrow \{1, \dots, n\}$ ;
2 while  $N \neq \emptyset$  do
3    $u^* \leftarrow$  cliente mais distante de 0 em  $N$ ;
4    $R \leftarrow [0, u^*, 0]$ ;  $N \leftarrow N \setminus \{u^*\}$ ;
5   repeat
6     para cada  $u \in N$  viável, calcular  $C_1(u)$  pela melhor posição;
7      $c_{\min} \leftarrow \min_u C_1(u)$ ;  $c_{\max} \leftarrow \max_u C_1(u)$ ;
8      $\text{RCL} \leftarrow \{u : C_1(u) \leq c_{\min} + \alpha(c_{\max} - c_{\min})\}$ ; // Eq. 2
9     if  $\text{RCL} \neq \emptyset$  then
10      sortear  $u$  uniformemente em RCL; inserir  $u$  em  $R$ ;  $N \leftarrow N \setminus \{u\}$ ;
11    end if
12  until  $\text{RCL} = \emptyset$ ;
13   $S \leftarrow S \cup \{R\}$ ;
14 end while
15 return  $S$ ;
```

Atualização reativa. Sejam $\bar{c}_i$ a média dos custos das soluções obtidas com α_i no bloco corrente e c^* o custo da

incumbente S^* . As probabilidades são recalculadas como

$$q_i = \left(\frac{c^*}{\bar{c}_i} \right)^\gamma, \quad p_i = \frac{q_i}{\sum_j q_j}, \quad (6)$$

seguindo o esquema de Prais and Ribeiro (2000). Quando um α_i produz soluções inviáveis no bloco, atribui-se a $\bar{c}_i$ um valor penalizado $1,25c^*$, mantendo a probabilidade baixa, mas não nula.

3.6 Fase 4: Busca Tabu

A Busca Tabu (Algoritmo 6) opera sobre a mesma vizinhança granular do VND e usa política *best-improvement* (avalia todos os movimentos admissíveis a cada iteração). A memória de curto prazo é implementada como um vetor $\mathcal{T}[c]$ que armazena, para cada cliente c , a iteração em que o atributo deixa de ser tabu. O *tenure* aplicado a cada movimento é amostrado uniformemente em $[T_{\min}, T_{\max}]$; quando $T_{\min} = T_{\max}$, recupera-se o caso de *tenure* fixo. O critério de aspiração admite um movimento tabu se ele resultar em uma solução melhor que S^* no critério lexicográfico adotado.

Algoritmo 6: Busca Tabu (memória de curto prazo, com aspiração)

Entrada: Solução inicial S_0

Parâmetros: $T_{\min}, T_{\max}$, limites $I_{\max}$ (iterações) e L (estagnação)

Saída: Melhor solução encontrada S^*

```
1  $S \leftarrow S_0; \quad S^* \leftarrow S_0;$ 
2  $\mathcal{T}[c] \leftarrow 0$  para todo  $c$ ;  $\text{iter} \leftarrow 0$ ;  $\text{estag} \leftarrow 0$ ;
3 while  $\text{iter} < I_{\max}$  e  $\text{estag} < L$  do
4    $\text{iter} \leftarrow \text{iter} + 1$ ;
5    $\mathcal{M} \leftarrow \text{Vizinhos}(S, \mathcal{N}_1, \dots, \mathcal{N}_\ell);$  // vizinhança granular
6    $m^* \leftarrow \text{nulo}$ ;  $\text{melhorAdm} \leftarrow +\infty$ ;  $m^\dagger \leftarrow \text{nulo}$ ;  $\text{melhorTabu} \leftarrow +\infty$ ;
7   foreach  $m \in \mathcal{M}$  do
8      $c'_m \leftarrow f(S \oplus m)$ ;
9     if não  $\text{ÉTabu}(m, \mathcal{T}, \text{iter})$  ou  $\text{Aspirado}(m, S^*)$  then
10      if  $c'_m < \text{melhorAdm}$  then
11         $m^* \leftarrow m$ ;  $\text{melhorAdm} \leftarrow c'_m$ ;
12      end if
13      else
14        if  $c'_m < \text{melhorTabu}$  then
15           $m^\dagger \leftarrow m$ ;  $\text{melhorTabu} \leftarrow c'_m$ ;
16        end if
17      end if
18    end foreach
19    if  $m^* = \text{nulo}$  then
20       $m^* \leftarrow m^\dagger$ ; // aspiração por omissão
21    end if
22     $S \leftarrow S \oplus m^*$ ;
23    foreach cliente  $c$  envolvido em  $m^*$  do
24      sortear  $T \in [T_{\min}, T_{\max}]$ ;  $\mathcal{T}[c] \leftarrow \text{iter} + T$ ;
25    end foreach
26    if  $f(S) \prec f(S^*)$  then
27       $S^* \leftarrow S$ ;  $\text{estag} \leftarrow 0$ ;
28    else
29       $\text{estag} \leftarrow \text{estag} + 1$ ;
30    end if
31 end while
32 return  $S^*$ ;
```

3.7 Operadores de Vizinhança

Oito operadores foram avaliados na fase de seleção de vizinhanças (Quadro 2). A avaliação de cada movimento utiliza cálculo Δ incremental e checagem de viabilidade simultânea de capacidade e janelas.

Quadro 2: Operadores de vizinhança avaliados

Operador	Tipo	Descrição do movimento
RelocateIntra	intra-rota	remove um cliente e o reinsere em outra posição da mesma rota
SwapIntra	intra-rota	troca a posição de dois clientes na mesma rota
2-opt intra	intra-rota	inverte um segmento da rota (eliminando cruzamentos)
Or-opt	intra/inter	desloca um segmento de comprimento 2 ou 3
RelocateInter	inter-rota	move um cliente de uma rota para outra
SwapInter	inter-rota	troca um cliente entre duas rotas
2-opt*	inter-rota	troca os sufixos finais de duas rotas
Cross-exchange	inter-rota	troca segmentos de comprimento 1 ou 2 entre duas rotas

3.8 Calibração por irace

Os hiperparâmetros do GRASP e da Busca Tabu foram calibrados com o pacote *irace* (López-Ibáñez et al., 2016), seguindo a metodologia de Birattari (2009). As instâncias foram divididas em dois conjuntos disjuntos: *input_tuning_set* (calibração) e *input_holdout_set* (validação final). O contrato de execução é idêntico para todas as configurações: mesmo *time-limit*, *max-iters* e *max-no-improve* desativados (apenas o tempo é orçamento), mesmo conjunto de vizinhanças no VND, mesmo μ e ε . O *score* minimizado é $\text{gap}_{\%} + \lambda \cdot \text{tempo}$ com $\lambda = 10^{-4}$ (priorizar gap, desempate por tempo). O Quadro 3 descreve os espaços de busca pré-registrados.

Quadro 3: Espaço de busca da calibração *irace* (pré-registrado)

Cenário	Parâmetro	Tipo	Faixa / valor
Tabu clássico	<i>tenure</i>	inteiro	[5, 50]
	<i>tabu-move-policy</i>	categórico	best (fixo)
GRASP fixo	<i>alpha</i>	real	[0,0, 1,0]
GRASP reativo	<i>reactive-update (b)</i>	inteiro	[5, 100]
	<i>reactive-alphas</i>	conjunto	{0,1;0,2;0,3;0,4;0,5}
	<i>reactive-gamma</i>	real	1,0 (fixo)

A política de fronteira segue López-Ibáñez et al. (2016): se o vencedor ficar na borda do intervalo, a faixa é expandida e a corrida é repetida. O orçamento inicial por cenário é *maxExperiments*=1000; a configuração final é congelada antes da execução no *holdout*.

3.9 Módulo IoT e Digital Twin

O módulo de simulação de sensores foi projetado para validar a integração entre monitoramento e replanejamento do solver, inspirado em Lozano et al. (2018) e Almeida and Borin (2021). O Quadro 4 resume seus aspectos funcionais.

Quadro 4: Aspectos funcionais do ambiente de simulação IoT

Componente	Descrição
Modelo de enchimento	Distribuição estatística com variação por período (pico/fora de pico)
Tecnologia de comunicação	LoRaWAN (simulada)
Critério de acionamento	Nível de enchimento acima de limiar configurável
Geração de demanda	Lixeiras ativas convertidas em clientes com janela de tempo
Interface com o solver	Envio de eventos ao módulo VRPTW a cada ciclo de monitoramento
Linguagem de implementação	Python; integração com solver C++ via interface de arquivos

O fluxo previsto é: (i) sensores reportam nível; (ii) ao ultrapassar o limiar, uma lixeira é convertida em um cliente VRPTW com janela; (iii) o conjunto de clientes ativos no ciclo é enviado ao solver; (iv) o solver replaneja e devolve as rotas ao Digital Twin para visualização. A integração completa entre simulador, solver e Digital Twin de visualização é parte das Etapas 6 e 7 do cronograma do subprojeto.

4 Experimentos e Resultados

4.1 Instâncias e Protocolo

Os experimentos utilizam as instâncias de Solomon de 100 clientes, divididas em famílias C (*clustered*), R (*random*) e RC (mista). Cada execução é avaliada sob os dois critérios discutidos em §3.2: DIMACS (distância, 1 casa decimal) e Solomon clássico (lexicográfico veículos–distância, 2 casas decimais). As BKS de referência para a comparação são as publicadas no sítio pessoal de Solomon, sob o critério clássico; quando disponíveis, valores DIMACS são reportados em paralelo.

4.2 Configuração Experimental

Hardware: processador Intel Core i7, 16 GB RAM, Linux Ubuntu 24.04. **Linguagem:** C++20 (solver) e Python (irace, simulador IoT, análise estatística). **Orçamento:** time-limit fixo por execução (mesmo valor para todos os métodos comparados); critérios secundários desativados. **Pipeline avaliado:**

1. Solomon I1 (referência construtiva)
2. Solomon I1 + VND
3. GRASP fixo (com VND)
4. GRASP reativo (com VND)
5. Solomon I1 + Busca Tabu

Métricas:

- $\text{Gap}_{\%} = \frac{\text{custo}_{\text{obtido}} - \text{custo}_{\text{BKS}}}{\text{custo}_{\text{BKS}}} \times 100$
- $\Delta_K = K_{\text{obtido}} - K_{\text{BKS}}$ (número de veículos)
- Tempo computacional (s) e número de iterações

4.3 Resultados Agregados por Família

A Tabela 1 apresenta o resultado médio por família e método, sob o critério Solomon clássico. Os valores serão consolidados na Etapa 7 do cronograma; o Quadro 5 sintetiza qualitativamente o perfil esperado de cada algoritmo, conforme observado em testes preliminares.

Tabela 1: Resultados agregados por família (a consolidar na Etapa 7)

Família	Método	Gap (%)	ΔK	Tempo (s)
C	Solomon I1	—	—	—
	I1 + VND	—	—	—
	GRASP fixo	—	—	—
	GRASP reativo	—	—	—
	Busca Tabu	—	—	—
R	Solomon I1	—	—	—
	I1 + VND	—	—	—
	GRASP fixo	—	—	—
	GRASP reativo	—	—	—
	Busca Tabu	—	—	—
RC	Solomon I1	—	—	—
	I1 + VND	—	—	—
	GRASP fixo	—	—	—
	GRASP reativo	—	—	—
	Busca Tabu	—	—	—

Quadro 5: Perfil qualitativo dos algoritmos implementados

Algoritmo	Perfil observado	Aplicação recomendada
Solomon I1	Construção rápida e determinística	Solução inicial; referência de comparação
VND	Refinamento local com baixo tempo de resposta	Cenários com restrição temporal estrita
GRASP fixo	Melhoria consistente com maior esforço computacional	Planejamento com orçamento de tempo moderado
GRASP reativo	Robustez frente a instâncias heterogêneas	Instâncias variadas sem calibração manual de α
Busca Tabu	Boa qualidade com tempo intermediário	Replanejamento mais frequente

4.4 Análise dos Operadores de Busca Local

A escolha das vizinhanças do VND seguiu uma análise de contribuição individual sobre o conjunto `tuning_set`. O Quadro 6 sintetiza o papel observado de cada grupo de operadores; movimentos entre rotas mostraram contribuição mais consistente nas melhorias de maior magnitude, enquanto operadores intra-rota atuam como refinamento fino.

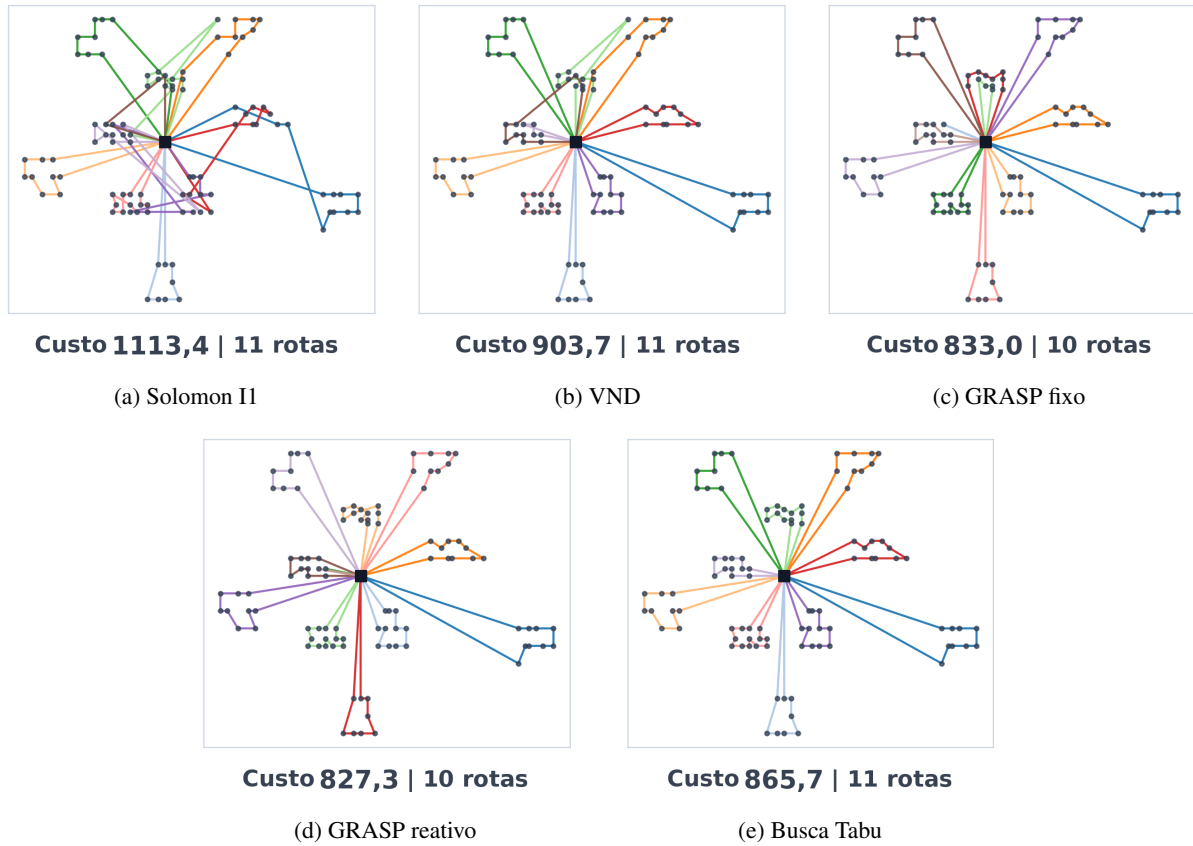
Quadro 6: Contribuição qualitativa dos grupos de operadores no VND

Grupo	Contribuição observada	Papel predominante
Movimentos entre rotas (Relocate inter, 2-opt*)	Maior impacto nas melhorias relevantes	Redistribuição global de clientes
Movimentos intra-rota (Relocate intra, 2-opt intra)	Contribuição em ajustes finos	Refinamento da sequência de atendimento
Operadores de troca (Swap intra, Swap inter)	Efeito complementar	Correções pontuais
Operadores complexos (Or-opt, Cross-exchange)	Uso mais localizado	Diversificação da busca

4.5 Comparação Visual em Instância Representativa (C102)

A Figura 1 compara, sob o critério DIMACS, as rotas produzidas por diferentes métodos na instância C102. Cada painel exibe, no rodapé, custo total e número de rotas. Entre os métodos avaliados nesta visualização preliminar, o GRASP reativo apresentou o melhor resultado (827,3 com 10 rotas).

Figura 1: Comparação de rotas na instância C102



Fonte: produção da própria autora.

4.6 Validação Estatística

A comparação entre algoritmos segue o protocolo não-paramétrico recomendado por Demšar (2006): teste de Friedman sobre os *ranks* de cada método nas instâncias do *holdout set*, seguido pelo *post-hoc* de Nemenyi com correção para múltiplas comparações. O resultado final inclui (i) o *p*-valor do Friedman, (ii) os *ranks* médios por método e (iii) o diagrama de diferenças críticas (CD diagram). A consolidação desses indicadores será apresentada na versão final após o encerramento da Etapa 7 (testes computacionais), conforme cronograma.

4.7 Integração IoT-Solver: Ensaio Inicial

A versão inicial do simulador IoT confirmou a geração de eventos de enchimento e a comunicação com o solver via interface de arquivos. O ensaio executado verifica: (i) criação consistente de instâncias VRPTW a partir do estado dos sensores; (ii) chamada do solver com orçamento limitado de tempo; (iii) retorno das rotas ao simulador para visualização. A integração completa com o Digital Twin (Etapa 6) e a validação em escala (Etapa 7) são as próximas metas do subprojeto.

5 Conclusão e Trabalhos Futuros

Este relatório apresentou a metodologia consolidada para o desenvolvimento de um *solver* VRPTW com integração IoT, aplicável à coleta de resíduos sólidos. Foram detalhados, em pseudocódigo formal, os algoritmos implementados em C++20: Solomon II, VND, GRASP (fixo e reativo) e Busca Tabu. A metodologia experimental adota duas convenções de objetivo (DIMACS e Solomon clássico) em paralelo, com BKS provenientes do sítio pessoal de Solomon devido à indisponibilidade do repositório Gehring & Homberger. A calibração por irace e a validação estatística não-paramétrica garantem rigor metodológico na comparação entre métodos.

As próximas etapas do subprojeto consistem em: (a) consolidar os experimentos sobre o conjunto holdout após calibração final via irace; (b) integrar o simulador IoT ao Digital Twin de visualização; (c) avaliar cenários mais representativos da operação real de coleta. Como extensões futuras, destacam-se a incorporação de metaheurísticas baseadas em população (HGS (Vidal et al., 2013)) e a utilização de dados reais para validação do modelo.

Referências

- Almeida, L. G. d. and Borin, J. F. (2021). Plataforma inteligente e sustentável para coleta de resíduos baseada em IoT e LPWAN. In *Anais do Workshop de Computação Urbana (COURB)*, pages 154–167. SBC.
- Barth, L., Schweiger, L., Benedech, R., and Ehrat, M. (2023). From data to value in smart waste management: Optimizing solid waste collection with a digital twin-based decision support system. *Decision Analytics Journal*, 9:100347.
- Birattari, M. (2009). *Tuning Metaheuristics: A Machine Learning Perspective*, volume 197 of *Studies in Computational Intelligence*. Springer.
- Bräysy, O. and Gendreau, M. (2005). Vehicle routing problem with time windows, part I: Route construction and local search algorithms. *Transportation Science*, 39(1):104–118.
- Cordeau, J.-F., Laporte, G., and Mercier, A. (2001). A unified tabu search heuristic for vehicle routing problems with time windows. *Journal of the Operational Research Society*, 52(8):928–936.
- Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30.
- Feo, T. A. and Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109–133.
- Gehring, H. and Homberger, J. (1999). A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In Miettinen, K., Mäkelä, M., and Toivanen, J., editors, *Proceedings of EUROGEN99*, pages 57–64. University of Jyväskylä.
- Gendreau, M. and Potvin, J.-Y. (2010). Tabu search. In Gendreau, M. and Potvin, J.-Y., editors, *Handbook of Metaheuristics*, volume 146 of *International Series in Operations Research & Management Science*, pages 41–59. Springer, 2 edition.
- Glover, F. (1989). Tabu search – part I. *ORSA Journal on Computing*, 1(3):190–206.
- Glover, F. (1990). Tabu search – part II. *ORSA Journal on Computing*, 2(1):4–32.
- Glover, F. and Laguna, M. (1997). *Tabu Search*. Kluwer Academic Publishers, Boston.

- Golden, B., Raghavan, S., and Wasil, E., editors (2008). *Vehicle Routing: Latest Advances and New Challenges*, volume 43 of *Operations Research/Computer Science Interfaces Series*. Springer. Based on the 12th DIMACS Challenge on Vehicle Routing Problems.
- Hansen, P. and Mladenović, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research*, 130(3):449–467.
- Lin, S. and Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516.
- López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., and Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58.
- Lozano, Á., Caridad, J., De Paz, J. F., Villarrubia Gonzalez, G., and Bajo, J. (2018). Smart waste collection system with low consumption LoRaWAN nodes and route optimization. *Sensors*, 18(5):1465.
- Melo, R. A., Nogueira, L. H., and Souza, S. R. (2008). Variable neighborhood descent with combinatorial neighborhood structures for the vehicle routing problem with time windows. *Pesquisa Operacional*, 28(3):483–500.
- Or, I. (1976). *Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking*. PhD thesis, Northwestern University.
- Pardini, K., Rodrigues, J. J. P. C., Diallo, O., Das, A. K., de Albuquerque, V. H. C., and Kozlov, S. A. (2020). A smart waste management solution geared towards citizens. *Sensors*, 20(8):2380.
- Potvin, J.-Y. and Rousseau, J.-M. (1995). An exchange heuristic for routeing problems with time windows. *Journal of the Operational Research Society*, 46(12):1433–1446.
- Prais, M. and Ribeiro, C. C. (2000). Reactive GRASP: An application to a matrix decomposition problem in TDMA traffic assignment. *INFORMS Journal on Computing*, 12(3):164–176.
- Ramson, S. R. J., Vishnu, S., Kirubaraj, A. A., Anagnostopoulos, T., and Abu-Mahfouz, A. M. (2021). A LoRaWAN IoT enabled trash bin level monitoring system. *IEEE Transactions on Industrial Informatics*, 17(8):5697–5707.
- Resende, M. G. C. and Ribeiro, C. C. (2003). Greedy randomized adaptive search procedures. In Glover, F. and Kochenberger, G. A., editors, *Handbook of Metaheuristics*, pages 219–249. Springer, Boston.
- Shaw, P. (1998). Using constraint programming and local search methods to solve vehicle routing problems. In *Principles and Practice of Constraint Programming – CP 1998*, volume 1520 of *Lecture Notes in Computer Science*, pages 417–431. Springer.
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265.
- Toth, P. and Vigo, D. (2003). The granular tabu search and its application to the vehicle-routing problem. *INFORMS Journal on Computing*, 15(4):333–346.
- Vidal, T., Crainic, T. G., Gendreau, M., and Prins, C. (2013). A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. *Computers & Operations Research*, 40(1):475–489.